

# Haibo Wang

Email: [haibo.wang@mail.concordia.ca](mailto:haibo.wang@mail.concordia.ca)

Google Scholar: <https://scholar.google.com/citations?user=BwpU6EIAAAAJ&hl=en>

## Highlights of Qualifications

- Well-cited top-tier publications (FSE, ASE, PLDI, TOSEM) across three fields in Computer Science (Software Engineering, Programming Languages, and AI).
- Diverse education experience across 3 countries (Canada, United Kingdom, China).
- Open-source contributions: revealed more than 200 bugs in popular open-source projects like Eclipse, IntelliJ IDEA, and Google V8 JavaScript engine.
- Runner-up for the 2023 Natural Language-Based Software Engineering Tool Competition.

## Education

### Concordia University, Canada

2023.09-Now

Ph.D. candidate (GPA 3.9/4.3), expected graduation time 2027.01

Computer Science

Supervisor: [Shin Hwei Tan](#), [shinhwei.tan@concordia.ca](mailto:shinhwei.tan@concordia.ca)

### Southern University of Science and Technology, China

2021.11-2023.08

Joint Ph.D. Program of Southern University of Science and Technology and University of Leeds (relocate to Concordia University)

### University of Leeds, United Kingdom

2020.09-2021.10

Joint Ph.D. Program of Southern University of Science and Technology and University of Leeds

Computer Science

Supervisor: [Shin Hwei Tan](#), [shinhwei.tan@concordia.ca](mailto:shinhwei.tan@concordia.ca); [Zheng Wang](#), [z.wang5@leeds.ac.uk](mailto:z.wang5@leeds.ac.uk)

### Beijing University of Posts and Telecommunications, China

2016.09-2019.06

Master of Science in Engineering (GPA 81/100)

Computer Technology

### China University of Petroleum, China

2011.09-2015.06

Bachelor of Science in Engineering (GPA 82/100)

Software Engineering

## Employment

### Software R & D center of Agricultural Bank of China

2019.07-2020.08

Full Stack Software Development Engineer for Customer Service System

### Ant Group Co., Ltd. of Alibaba

2018.06-2018.09

Java Software Development Intern for Alipay Insurance System

## Research Interests

Automated Software Testing; Refactoring; AI for SE; Green SE; Human Aspects of SE

## Publications

### Preprints

(P1) Tan, Honghao, **Haibo Wang**, and Shin Hwei Tan. "Structured Safety Auditing for Balancing Code Correctness and Content Safety in LLM-Generated Code." arXiv preprint arXiv:2604.12088 (2026).

(P2) Tan, Shin Hwei, **Haibo Wang**, and Heng Li. "Ethics Testing: Proactive Identification of Generative AI System Harms." arXiv preprint arXiv:2604.22089 (2026).

### Conference Papers

(C1) Tan, Honghao, **Haibo Wang**, Dianyu Pressato, Yisen Xu, and Shin Hwei Tan. "Automated Harmfulness Testing for Code Large Language Models." 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2025. Core A\*, Acceptance rate 22%

(C2) **Wang, Haibo**, Zhuolin Xu, and Shin Hwei Tan. "Testing Refactoring Engine via Historical Bug Report driven LLM." 2nd ACM International Conference on AI Foundation Models and Software Engineering (FORGE), 2025. Acceptance rate 47%

(C3) **Wang, Haibo**, Zezhong Xing, Chengnian Sun, Zheng Wang, and Shin Hwei Tan. "Towards Diverse Program Transformations for Program Simplification." ACM International Conference on the Foundations of Software Engineering (FSE), 2025. Core A\*, Acceptance rate 21%

(C4) Win, Hsu Myat, **Haibo Wang**, and Shin Hwei Tan. "Towards automated detection of unethical behavior in open-source software projects." ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2023. Core A\*, Acceptance rate 21%.

(C5) Ye, Guixin, Zhanyong Tang, Shin Hwei Tan, Songfang Huang, Dingyi Fang, Xiaoyang Sun, Lizhong Bian, **Haibo Wang**, and Zheng Wang. "Automated conformance testing for JavaScript engines via deep compiler fuzzing." 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI), 2021. Core A\*, Acceptance rate 20%, current citation 96

### Journal Articles

(J1) **Wang, Haibo**, Zhuolin Xu, Huaian Zhang, Nikolaos Tsantalis, and Shin Hwei Tan. "An Empirical Study of Refactoring Engine Bugs." ACM Transactions on Software Engineering and Methodology (TOSEM), 2025. Impact factor 6.2 (2024)

### Workshop Papers

(W1) Li, Ying, **Haibo Wang**, Huaian Zhang, and Shin Hwei Tan. "Classifying code comments via pre-trained programming language model."

## **Contributions to Open-Source Community**

1. Our study and tool from paper J1 and C2 successfully revealed bugs (detected 152, submitted 40, confirmed 22, fixed 11) in modern IDEs like Eclipse, IntelliJ IDEA, and NetBeans. Following are three of our detected bugs, so as for the other works:

[Eclipse] Make static refactoring <https://github.com/eclipse-jdt/eclipse.jdt.ui/issues/1824>

[Eclipse] Pull up refactoring <https://github.com/eclipse-jdt/eclipse.jdt.ui/issues/1773>

[IntelliJ IDEA] Extract method refactoring <https://youtrack.jetbrains.com/issue/IDEA-354122>

2. As the outcomes of paper C3, we revealed eight bugs (all fixed) in a state of art refactoring detection tool RefactoringMiner.

[RefactoringMiner] Inline variable refactoring <https://github.com/tsantalis/RefactoringMiner/issues/678>

[RefactoringMiner] Invert condition refactoring <https://github.com/tsantalis/RefactoringMiner/issues/679>

[RefactoringMiner] Merge conditional refactoring <https://github.com/tsantalis/RefactoringMiner/issues/680>

3. Our tool from paper C5 successfully identified 158 unique JS engine bugs, and 115 have already been fixed by the developers.

[V8] <https://issues.chromium.org/issues/42212638>

[ChakraCore] <https://github.com/chakra-core/ChakraCore/issues/6278>

[JavaScriptCore] [https://bugs.webkit.org/show\\_bug.cgi?id=201910](https://bugs.webkit.org/show_bug.cgi?id=201910)

---

## **Research Projects**

### **Coverage-based Harmfulness Testing for LLM Code Transformation**

- Novel coverage-guided harmfulness testing framework that automatically synthesizes prompts using a set of prompt templates injected with diverse harmful keywords to perform various types of transformations on a set of mined benign programs.

- GitHub repository: <https://github.com/honghao-tan/CHT>

- Our evaluations of CHT on four Code LLMs and GPT-4o-mini show that content moderation in Code LLMs is relatively easy to bypass where LLMs may generate harmful keywords embedded within identifier names or code comments without giving any warning.

- As the second author, I am responsible for the experiments design, dataset collection, and academic writing.

### **Towards Understanding Refactoring Engine Bugs**

- The first systematic study of refactoring engine bugs by analyzing bugs arising in popular IDEs (i.e., Eclipse, IntelliJ IDEA, and NetBeans)

- GitHub repository: <https://github.com/HaiboWang1992/RETester>

- Our study revealed 134 new bugs in the latest version of IDEs. Among the 22 bugs we submitted, 11 bugs are confirmed by their developers, and seven of them have already been fixed.

- As the first author, I am responsible for the overall methodology design, experiments, academic writing, and artifact preparation.

### **Testing Refactoring Engine via Historical Bug driven LLM**

- An LLM-based framework for automated refactoring engine testing by using input program structure templates extracted from historical bug reports and input program characteristics that are error-prone, LLMs are prompted to perform refactoring-preserving transformations.

- GitHub repository: <https://github.com/HaiboWang1992/REBDetector>

- Our tool successfully revealed 18 previously unknown bugs in the latest version of Eclipse and IntelliJ IDEA, seven of them have been confirmed by their developers, and three have been fixed.

- As the first author, I am responsible for the overall methodology design, experiments, academic writing, and artifact preparation.

### **Towards Diverse Program Transformations for Program Simplification**

- The first study on how real-world developers simplify programs in open-source software projects. Based on our observations, we propose SimpT5, a tool to automatically produce simplified programs that are semantically equivalent programs with reduced lines of code.

- GitHub repository: <https://anonymous.4open.science/r/Automated-Program-Simplification-CCC6>

- We submitted eight bug reports to RefactoringMiner and all of them have been fixed. Our experimental results show that SimpT5 outperforms prior approaches in automating developer-induced program simplification.

- As the first author, I am responsible for the overall methodology design, experiments, academic writing, and artifact preparation.

---

## **Academic Services**

Sub-reviewer for ASE 2022, FSE 2022, ISSTA 2022, ICSE 2023, ISSTA 2023, FSE 2024, ICSE 2026.

Student volunteer for ICSE 2025 (20 hours), FSE 2025 (25 hours). My volunteer tasks including service at reception desk, manage poster presentation, coordinate presentation and banquet.

---

## **Teaching**

2021.09, TA for Software Engineering Course (CS304), Southern University of Science and Technology

- Manage and grade presentations, assignments for more than 100 students

- Coordinate weekly lab section for 2 hours per week

2022.01, TA for Frontiers of Computer Science Course (CS318), Southern University of Science and Technology

- Grading assignments for around 30 students

- Coordinate and answer questions at weekly class for 1 hours per week

---

### **Award/Scholarship**

2025.04, ACM SIGSOFT CAPS Award (travel award, \$1,250 CAD)

2025.01, Concordia University SGS Conference Award (travel award, \$1,750 CAD)

2023.09, Concordia University International Tuition Award of Excellence (\$44,893 CAD)

2016.09, Master Scholarship of Beijing University of Posts and Telecommunications (\$1,563 CAD)

2012.06, Science and Technology Innovation Scholarship of China University of Petroleum (\$1,172 CAD)

2023.05, Runner-up Award for the 2023 Natural Language-Based Software Engineering Tool Competition

---

### **Invited Talks**

2025.04, presentation at FORGE 2025: Testing Refactoring Engine via Historical Bug Report driven LLM

2025.06, presentation at FSE 2025: Towards Diverse Program Transformations for Program Simplification

---